



München Stadtrallye

Technische Bedienungsanleitung

Spieler-App & Operator-Leitstand — für das Orga-Team des Deutsch-Japanischen Sportjugend-Austauschs

Projekt	stadtrally-muc-26 (Firebase)
Spieler-App	index.html
Operator-Leitstand	operator.html
Zugangscode Leitstand	muc2026 (Datei: firebase-config.js)
Stand	Juli 2026

Inhalt

1. Überblick & Architektur
2. Dateien im Projekt
3. Konfiguration (firebase-config.js)
4. Deployment / Aktualisieren der App
5. Spieler-App im Detail (index.html)
6. Operator-Leitstand im Detail (operator.html)
7. Gruppen & Zugangscodes
8. Stationen & Koordinaten
9. Ablauf am Veranstaltungstag (Checkliste)
10. Fehlerbehebung
11. Datenschutz & Datenlöschung

1 Überblick & Architektur

Die Stadtrallye ist eine reine **Web-App ohne Build-Prozess**: statisches HTML/CSS/JavaScript, gehostet über **Firebase Hosting**. Für die Live-Synchronisation zwischen den Spielergruppen und dem Orga-Team läuft im Hintergrund eine **Firebase Realtime Database**.

Es gibt zwei getrennte Oberflächen, die dieselben Spieldaten und dieselbe Datenbank nutzen:

index.html	Die Spieler-App. Wird von jeder Gruppe auf dem eigenen Handy geöffnet (z. B. per QR-Code oder Link). Zeigt Route, Aufgaben, Stempel, Foto-Upload und das End-Rätsel.
operator.html	Der Leitstand für das Orga-Team. Zeigt den Live-Fortschritt aller Gruppen, eine Live-Karte der geteilten Standorte, Zugangscodes und Backup-PDFs zum Ausdrucken.

Datenfluss

Beide Seiten laden dieselbe Firebase-Konfiguration (**firebase-config.js**) und dieselben Spieldaten (**game-data.js**: Stationen, Gruppen, Lösungswort). Die Spieler-App schreibt Fortschritt, Fotos und optional den Standort in die Realtime Database; der Leitstand liest daraus live (*on('value', ...)*) und aktualisiert sich automatisch, ohne Neuladen der Seite.

```
Spieler-App (index.html)          Firebase Realtime Database          Leitstand (operator.html)
app.js --cloudPush--> rallye/<gruppe> { stamps, solved,
                                     stations, loc, updatedAt } --live--> operator.js
app.js --Foto-Upload--> rallye_photos/<gruppe>/s<n>_<i> { img (Base64), at }
```

Offline-Fallback

Fällt die Internetverbindung oder Firebase aus, funktioniert die Spieler-App trotzdem weiter: der komplette Spielstand wird zusätzlich im **localStorage** des Handys gespeichert (*rally_state_<gruppe>*). Der Cloud-Sync ist also ein Zusatz, kein Muss. Für den Notfall gibt es zudem druckbare Papier-Anleitungen (siehe Abschnitt 6.5).

2 Dateien im Projekt

Datei	Zweck
index.html	Spieler-App: komplettes Markup + eingebettetes CSS für alle Bildschirme (Start, Gruppenwahl, Route).
operator.html	Leitstand: Markup + CSS für Zugangs-Gate, Gruppen-Karten, Live-Karte (Leaflet).
app.js	Gesamte Spiellogik der Spieler-App: Zustand, Geofencing, Foto-Upload, Rätsel-Prüfung, Cloud-Sync.
operator.js	Logik des Leitstands: Login-Gate, Live-Rendering der Gruppenkarten, Karten-Marker, Wipe-Funktion.
game-data.js	Einzige Quelle der Spieldaten (Stationen, Koordinaten, Texte DE/JA, Gruppen, Zugangscodes, Lösungswort). Wird von app.js, operator.js und print.js gemeinsam genutzt.
print.js	Erzeugt die druckbare A4-Backup-Spielanleitung pro Gruppe (Papierversion für den Notfall).
firebase-config.js	Firebase-Zugangsdaten (öffentlich, unkritisch) + der geheime Operator-Zugangscode.
firebase.json	Firebase-Hosting-Konfiguration (welcher Ordner wird deployt).
database.rules.json	Zugriffsregeln der Realtime Database (aktuell: öffentlich lesbar/schreibbar, siehe Abschnitt 11).
.firebaseerc	Verknüpft den lokalen Projektordner mit dem Firebase-Projekt „stadtrally-muc-26“.

Cache-Busting bei Änderungen

In index.html und operator.html werden die Skripte mit einer Versionsnummer eingebunden, z. B. `app.js?v=5`. Wird eine Datei inhaltlich geändert (z. B. game-data.js oder app.js), muss die Zahl hinter `?v=` in der jeweiligen HTML-Datei erhöht werden — sonst liefern Handys mit gecachter alter Version weiterhin den alten Code aus.

3 Konfiguration (firebase-config.js)

Alle Zugangsdaten stehen zentral in **firebase-config.js**. Diese Datei wird von index.html und operator.html geladen.

3.1 Firebase-Projekt

Die Werte in `FIREBASE_CONFIG` stammen aus der Firebase-Konsole (Projekt-Einstellungen → „Meine Apps“ → Web-App → SDK-Konfiguration). Diese Werte sind bewusst öffentlich im Code sichtbar — das ist bei Firebase-Web-Apps normal und unkritisch, solange die **Datenbank-Regeln** (Abschnitt 11) sauber gesetzt sind.

```
const FIREBASE_CONFIG = {
  apiKey: "AIzaSyA1...",
  authDomain: "stadtrally-muc-26.firebaseio.com",
  databaseURL: "https://stadtrally-muc-26-default-rtdb.europe-west1.firebaseio.com",
  projectId: "stadtrally-muc-26", ...
};
```

3.2 Operator-Zugangscode

Der Zugang zum Leitstand (operator.html) ist mit einem einfachen Code geschützt — **kein echtes Login, nur Zugriffsschutz** vor Zufallsbesuchern:

```
const OPERATOR_CODE = "muc2026";
```

Der Code wird nach Eingabe in `sessionStorage` gemerkt (Schlüssel `op_ok`) — bleibt also nur für die aktuelle Browser-Sitzung gespeichert. Über den Button „Sperren“ oben rechts im Leitstand wird er entfernt.

Empfehlung: Diesen Code vor dem Event ändern (eigener, nicht erratbarer Wert) und nur ans Orga-Team weitergeben — er steht im Klartext im JavaScript-Quellcode und ist technisch kein echter Zugriffsschutz auf die Daten selbst.

4 Deployment / Aktualisieren der App

Die App liegt als statische Website auf **Firebase Hosting**, Projekt `stadtrally-muc-26`. Änderungen werden über die Firebase-CLI veröffentlicht.

4.1 Voraussetzungen (einmalig)

```
npm install -g firebase-tools
firebase login
```

4.2 Neue Version veröffentlichen

Im Projektordner (dort, wo `firebase.json` liegt):

```
firebase deploy
```

Das lädt alle Dateien im Projektordner gemäß `firebase.json` hoch (Hosting) und aktualisiert bei Bedarf auch die Datenbank-Regeln aus `database.rules.json`. Nach dem Deploy sind die Änderungen sofort live.

4.3 Nur die Datenbank-Regeln aktualisieren

```
firebase deploy --only database
```

4.4 Wichtig bei Text-/Daten-Änderungen

- Änderungen an Stationen, Texten, Gruppen oder Zugangscodes erfolgen zentral in **game-data.js** — beide Oberflächen und die PDF-Anleitungen ziehen von dort automatisch mit.
- Nach jeder inhaltlichen Änderung an `app.js` / `operator.js` / `game-data.js` / `print.js` die Versionsnummer `?v=` in `index.html` bzw. `operator.html` erhöhen (siehe Abschnitt 2) und erneut deployen.
- Vor dem eigentlichen Event: Testspiel (Abschnitt 6.4) und Leitstand einmal komplett durchklicken.

5 Spieler-App im Detail (index.html)

5.1 Sprache

Umschaltbar zwischen Deutsch und Japanisch oben rechts (DE / 日本語). Die Wahl wird in `localStorage` gespeichert und bleibt beim nächsten Öffnen erhalten. Alle Texte sind zweisprachig in `game-data.js` hinterlegt.

5.2 Gruppenwahl & Zugangscode

Es gibt 5 Gruppen mit je einem 4-stelligen Zugangscode (Tabelle in Abschnitt 7). Wählt eine Person in der App eine Gruppe, erscheint ein Code-Eingabefeld. Ist der Code richtig, wird die Freischaltung lokal gemerkt (`rally_code_ok_<gruppe>`), sodass beim erneuten Öffnen kein Code mehr nötig ist. Jede Gruppe durchläuft dieselben 6 Stationen, aber in unterschiedlicher Reihenfolge (siehe `order` je Gruppe in `game-data.js`).

5.3 Standort-Freischaltung von Stationen (Geofencing)

Aufgaben einer Station werden erst sichtbar, wenn sich die Gruppe der GPS-Position der Station auf **200 m** nähert (Konstante `UNLOCK_RADIUS_M` in `app.js`). Solange eine gesperrte Station geöffnet ist, läuft im Hintergrund `navigator.geolocation.watchPosition()` und berechnet die Entfernung per Haversine-Formel. Die GPS-Ungenauigkeit wird berücksichtigt: es wird ein Sicherheitsabstand (die gemeldete Genauigkeit, gedeckelt auf 50 m) von der Distanz abgezogen, bevor entriegelt wird. Sobald eine Station entriegelt ist, bleibt sie es dauerhaft (im lokalen Spielstand gespeichert) — die Gruppe kann sich also wieder entfernen, ohne die Aufgabe erneut zu verlieren.

Das Geofencing benötigt vom Handy die Erlaubnis für Standortzugriff. Ohne diese Erlaubnis bleiben Stationen gesperrt (Fehlermeldung „Standort-Zugriff nötig – bitte erlauben“) — bei der Einweisung der Gruppen unbedingt auf diese Berechtigung hinweisen.

5.4 Standort mit dem Orga-Team teilen (optional)

Getrennt vom Geofencing gibt es einen Schalter „Standort fürs Orga-Team teilen“. Ist er aktiv, sendet die App den Live-Standort laufend an `rallye/<gruppe>/loc` in der Datenbank — dieser erscheint dann auf der Live-Karte im Leitstand. Der Schalter ist freiwillig und jederzeit von den Spieler:innen abschaltbar; beim Ausschalten wird der gespeicherte Standort sofort aus der Datenbank entfernt.

5.5 Aufgaben, Stempel & Foto-Upload

Jede Station hat mehrere Aufgaben, teils mit Foto-Pflicht. Text-Aufgaben werden per Antippen abgehakt. Foto-Aufgaben öffnen die Handy-Kamera (`<input type=file capture=environment>`); das Bild wird im Browser auf max. 900 px verkleinert und als JPEG (Qualität 70 %) direkt als Base64-Daten in `rallye_photos/<gruppe>/s<Station>_<Aufgabe>` in der Datenbank gespeichert — es gibt keinen separaten Datei-Speicher (kein Firebase Storage). Nach Erledigung aller sichtbaren Schritte gibt der „Stempel holen“-Button einen Buchstaben frei, der im End-Rätsel gebraucht wird.

5.6 Das End-Rätsel

Aus den 6 gesammelten Stempel-Buchstaben ergibt sich das Lösungswort `KIZUNA` (japanisch 絆, „Verbundenheit“), hinterlegt als `SOLUTION` in `game-data.js`. Die Prüfung erfolgt rein im Browser (Groß-/Kleinschreibung und Leerzeichen werden ignoriert).

5.7 Fortschritt zurücksetzen

Der Button „Fortschritt zurücksetzen“ auf dem Routen-Bildschirm löscht (nach Sicherheitsabfrage) nur den lokalen Spielstand dieser Gruppe auf diesem Gerät. Der zuletzt an die Datenbank gemeldete Stand im Leitstand bleibt bestehen, bis die Gruppe erneut etwas abhakt oder stempelt.

5.8 Testmodus für Spieler-Vorschau

Ruft man die App mit dem Zusatz `?test=1` auf, z. B.:

```
https://stadtrally-muc-26.web.app/index.html?test=1
```

startet der Testmodus (rot-oranges Banner „Testmodus“ oben):

- Geofencing ist deaktiviert — alle Stationen sind sofort ohne Standortabgleich offen.
- Kein Gruppen-Zugangscode nötig.
- Kein Cloud-Sync: der Fortschritt wird nur lokal im Browser gespeichert und erscheint **nicht** im Operator-Leitstand.
- Der Standort-Teilen-Schalter wird ausgeblendet, da er im Testmodus ohnehin wirkungslos wäre.

Der Testmodus bleibt für die restliche Browser-Sitzung aktiv (`sessionStorage rally_test`), auch nach einem Neuladen der Seite ohne den Parameter — bis der Tab/Browser vollständig geschlossen wird. Praktisch zum Durchklicken der ganzen Rallye vor dem Event, ohne tatsächlich an jede Station laufen zu müssen.

6 Operator-Leitstand im Detail (operator.html)

6.1 Zugang

operator.html öffnen, den Zugangscode aus firebase-config.js eingeben (Standard ab Werk: [muc2026](#)). Der Code gilt nur für diese Browser-Sitzung; „Sperren“ oben rechts meldet ab.

6.2 Gruppen-Übersicht

Für jede der 5 Gruppen zeigt eine Karte in Echtzeit:

- Aktivitätsstatus: „noch nicht gestartet“ oder „aktiv · vor X Min“ — wird rötlich markiert, wenn seit über 15 Minuten kein Update mehr einging (mögliches Problem mit Verbindung/Akku).
- Fortschrittsbalken und Stempelzähler (X / 6).
- 6 Stationsbuchstaben-Kacheln — leuchten auf, sobald die jeweilige Station gestempelt wurde.
- Rätsel-Status: „Rätsel offen“ oder „Rätsel gelöst“.
- Standortstatus: „Standort live · vor X Min“, falls die Gruppe die Standortfreigabe aktiviert hat, sonst „kein Standort“.
- Zugangscode der Gruppe (zum Weitergeben, falls jemand ihn vergessen hat).
- Button „Spielanleitung als PDF (A4)“ — siehe 6.5.

6.3 Live-Karte

Unterhalb der Gruppenkarten zeigt eine Karte (Leaflet + OpenStreetMap-Kacheln) die aktuellen Standorte aller Gruppen, die die Standortfreigabe aktiviert haben, als farbige Marker mit Gruppen-Emoji. Die Karte zentriert sich automatisch auf alle sichtbaren Gruppen. Ohne aktive Freigabe erscheint keine Gruppe auf der Karte — das ist gewollt (Datenschutz, siehe Abschnitt 11).

6.4 Testspiel-Link

Die Kachel „Testspiel starten“ öffnet [index.html?test=1](#) in einem neuen Tab — der in Abschnitt 5.8 beschriebene Testmodus. Damit kann das Orga-Team die komplette Rallye vorab durchklicken, ohne Zugangscode einzugeben oder Geofencing/GPS zu benötigen, und ohne dass dieser Testlauf als „echte“ Gruppe im Leitstand auftaucht.

6.5 Backup-Spielanleitung als PDF drucken

Über „Spielanleitung als PDF (A4)“ auf einer Gruppenkarte öffnet print.js eine druckfertige A4-Seite mit allen Stationen dieser Gruppe in der richtigen Reihenfolge, zweisprachig, inklusive Zugangscode, Ankreuzfeldern für Aufgaben und Stempel-Kästchen — aber **ohne** die Lösung des End-Rätsels (die wird bewusst nur in der App geprüft, um nicht zu spoilern). Im sich öffnenden Druckdialog „Als PDF speichern“ wählen. Diese Ausdrücke sind als **Papier-Backup** gedacht, falls Handy, Akku oder Internetverbindung einer Gruppe während der Rallye ausfallen.

Empfehlung: Für jede Gruppe schon vor dem Event einmal die PDF-Anleitung erzeugen und ausdrucken, griffbereit beim Orga-Team.

6.6 Alle Daten löschen (Wipe)

Der rote Button „Alle Rallye-Daten löschen“ entfernt nach Sicherheitsabfrage **unwiderruflich** die kompletten Pfade `rallye/` (Fortschritt, Standorte) und `rallye_photos/` (alle hochgeladenen Fotos) aus der Firebase-Datenbank.

Nur nach dem Event benutzen! Es gibt keine Rückgängig-Funktion und kein automatisches Backup dieser Daten. Vorher bei Bedarf Fotos/Fortschritt sichern (z. B. Screenshots des Leitstands).

7 Gruppen & Zugangscodes

Jede Gruppe hat einen eigenen 4-stelligen Zugangscode und eine eigene Stationsreihenfolge (Startstation unterschiedlich, damit sich die Gruppen an den Stationen nicht stauen).

Gruppe	Farbe	Zugangscode	Reihenfolge der Stationen (1→6)
Löwen		1860	Viktualienmarkt → Hofbräuhaus → Eisbachwelle → Odeonsplatz → Frauenkirche → Asamkirche
Adler		1900	Hofbräuhaus → Eisbachwelle → Odeonsplatz → Frauenkirche → Asamkirche → Viktualienmarkt
Eisbach		2020	Eisbachwelle → Odeonsplatz → Frauenkirche → Asamkirche → Viktualienmarkt → Hofbräuhaus
Olympia		1972	Odeonsplatz → Frauenkirche → Asamkirche → Viktualienmarkt → Hofbräuhaus → Eisbachwelle
Isar		2026	Frauenkirche → Asamkirche → Viktualienmarkt → Hofbräuhaus → Eisbachwelle → Odeonsplatz

Lösungswort des End-Rätsels (für alle Gruppen gleich): **KIZUNA** (絆).

Änderbar zentral in game-data.js unter **GROUPS** (Feld **code**) — Codes hier und im Code müssen identisch bleiben mit den in operator.js hinterlegten **GROUP_META**-Werten.

8 Stationen & Koordinaten

#	Buchstabe	Station	Koordinaten (lat, lng)
1	K	Viktualienmarkt	48.13512, 11.57619
2	I	Hofbräuhaus	48.13746, 11.58062
3	Z	Eisbachwelle	48.14593, 11.59076
4	U	Odeonsplatz & Feldherrnhalle	48.14222, 11.57646
5	N	Frauenkirche	48.13853, 11.57342
6	A	Asamkirche	48.13289, 11.56850

Startpunkt für alle Gruppen: **Marienplatz** (Fischbrunnen). Die 200-m-Freischaltradien (Abschnitt 5.3) beziehen sich auf genau diese Koordinaten. Änderungen an Namen, Texten, Aufgaben oder Koordinaten erfolgen zentral im Array `STATIONS` in `game-data.js`.

9 Ablauf am Veranstaltungstag (Checkliste)

Vorher

- Testspiel (index.html?test=1) einmal komplett durchklicken — alle Texte, Fotos, Rätsel prüfen.
- Operator-Leitstand öffnen, Zugangscode testen, Live-Karte prüfen.
- Für jede Gruppe die PDF-Backup-Anleitung erzeugen und ausdrucken (Abschnitt 6.5).
- Sicherstellen, dass Realtime Database vorher geleert ist (falls von einem früheren Test noch Daten drinstehen: „Alle Rallye-Daten löschen“, Abschnitt 6.6).
- Gruppen-Zugangscode an die jeweiligen Gruppen ausgeben.
- Prüfen, dass alle Handys mobile Daten/WLAN und Standortzugriff im Browser erlauben können.

Während der Rallye

- Leitstand offen halten (Tablet/Laptop) und Fortschritt sowie „vor X Min“-Zeitstempel im Auge behalten.
- Bei „stale“ (rötlich, seit >15 Min keine Aktivität): Gruppe kontaktieren.
- Standort erscheint nur, wenn eine Gruppe die Freigabe aktiv eingeschaltet hat — kein Standort ist kein Fehler, sondern der Normalfall bei ausgeschalteter Freigabe.
- Bei technischen Problemen einer Gruppe: Papier-Backup (PDF) aushändigen, Rallye kann ohne App fortgesetzt werden.

Danach

- Bei Bedarf Fotos aus dem Leitstand/der Datenbank sichern.
- „Alle Rallye-Daten löschen“ im Leitstand ausführen (Abschnitt 6.6 & 11) — Pflicht aus Datenschutzgründen, da die Datenbank ohne Login öffentlich lesbar ist.

10 Fehlerbehebung

Symptom	Ursache / Lösung
Station bleibt gesperrt trotz Vor-Ort-Sein	Standortzugriff im Browser nicht erlaubt, oder GPS-Genauigkeit schlecht (Innenräume/Häuserschluchten). Prüfen: Freigabemeldung „Standort wird geprüft...“ vs. Fehlermeldung. Ggf. draußen/im Freien neu versuchen.
Gruppe erscheint nicht im Leitstand	Gruppe wurde noch nicht ausgewählt/gestartet, oder läuft im Testmodus (?test=1 — der pusht bewusst nicht in die Cloud).
Kein Standort auf der Live-Karte	Standort-Freigabe-Schalter in der App wurde von der Gruppe nicht aktiviert. Ist freiwillig, keine technische Störung.
Foto-Upload schlägt fehl	Keine Internetverbindung → Fortschritt wird lokal als „gespeichert (offline)“ markiert, Foto-Upload zur Datenbank erst bei erneuter Verbindung erneut versuchen (Aufgabe erneut anhaken/Foto neu wählen).
„Keine Firebase-Config“ im Leitstand	firebase-config.js fehlt, ist nicht korrekt eingebunden, oder FIREBASE_CONFIG.apiKey beginnt noch mit Platzhaltertext „DEIN...“.
Änderungen erscheinen nicht auf den Handys	Browser-Cache. Versionsnummer ?v= der betroffenen Skripte in index.html/operator.html erhöhen und neu deployen (Abschnitt 2, 4).
Falscher Operator-Zugangscode	Wert aus firebase-config.js (OPERATOR_CODE) prüfen — Groß-/Kleinschreibung wird exakt verglichen.
Falscher Gruppen-Code	Codes exakt wie in Abschnitt 7 / game-data.js hinterlegt (4 Ziffern, kein Leerzeichen).

11 Datenschutz & Datenlöschung

Wichtig: Die Realtime-Database-Regeln (database.rules.json) stehen aktuell auf `.read: true / .write: true` für die Pfade `rallye` und `rallye_photos`. Das heißt: Wer die databaseURL kennt, kann Fortschritt, Live-Standorte und alle hochgeladenen Fotos ohne Login lesen (und theoretisch auch verändern). Das ist eine bewusste Vereinfachung für ein kurzes, einmaliges Event ohne Nutzerkonten — erfordert aber sorgfältigen Umgang.

Daraus folgt für den Betrieb:

- Die databaseURL/Firebase-Konfiguration nicht unnötig verbreiten, obwohl sie technisch im Quellcode öffentlich ist.
- Standort-Freigabe ist in der Spieler-App bewusst **opt-in** (Schalter, Standard aus) und jederzeit abschaltbar.
- Nach dem Event: **zwingend** „Alle Rallye-Daten löschen“ im Leitstand ausführen (Abschnitt 6.6), um Fotos und Standortverläufe zu entfernen.
- Der Operator-Zugangscode schützt nur den Leitstand-Bildschirm, nicht die Datenbank selbst — er ersetzt keine echte Authentifizierung.

Ende der technischen Anleitung · München Stadtrallye · Deutsch-Japanischer Sportjugend-Simultanaustausch